

Qurio Project Context

Project Overview

Qurio is a personal digital garden built with Next.js.

It documents experiments, builds, notes, journal entries, and lessons learned through hands-on building. The site is designed around the idea that curious people learn by building, reflecting, and sharing what they discover.

Qurio is not a polished expert platform. It is a practical learning archive that captures the process of figuring things out in public.

The core philosophy is:

Learn → Build → Document → Improve

Purpose

Qurio exists to turn curiosity into action.

The site should help document:

- Questions being explored
- Experiments being tested
- Projects being built
- Lessons learned
- Technical notes
- Personal reflections
- Practical discoveries

The goal is to create a growing library of useful learning artifacts rather than scattered notes across chats, files, and unfinished ideas.

Brand Direction

Qurio should feel:

- Curious
- Practical
- Honest
- Calm

- Thoughtful
- Experimental
- Builder-oriented
- Anti-hype
- Reading-first

The intended audience reaction is:

“Let me try that.”

Not:

“That person is an expert.”

The site should encourage action, confidence, and self-directed learning.

Tech Stack

Qurio currently uses:

- Next.js 16
- React 19
- TypeScript
- MDX through `next-mdx-remote`
- `gray-matter` for frontmatter parsing
- `remark-gfm` for GitHub-flavored Markdown
- Static export build output

The site is content-driven. Most entries are written as local `.mdx` files and rendered into static pages.

Core Repository Structure

```
qurio/
├─ app/
│   ├─ layout.tsx
│   ├─ page.tsx
│   ├─ about/page.tsx
│   ├─ now/page.tsx
│   ├─ notes/page.tsx
│   ├─ notes/[slug]/page.tsx
│   ├─ experiments/page.tsx
│   ├─ experiments/[slug]/page.tsx
│   └─ builds/page.tsx
```

```
|   └─ builds/[slug]/page.tsx
├─ components/
|   ├── Header.tsx
|   ├── Footer.tsx
|   ├── PageHeader.tsx
|   ├── ContentCard.tsx
|   └─ corresponding `.module.css` files
├─ content/
|   ├── now/current.mdx
|   ├── notes/
|   ├── experiments/
|   ├── builds/
|   └─ journal/
├─ lib/
|   └─ content.ts
├─ styles/
|   └─ variables.css
├─ public/
|   └─ images/
├─ docs/
|   ├── why-quirio-exists.md
|   ├── principles.md
|   └─ roadmap.md
├─ package.json
├─ next.config.mjs
├─ tsconfig.json
└─ README.md
```

Architecture

Quirio follows a simple file-based content architecture.

1. Content Layer

Content lives in the `content/` directory as local `.mdx` files.

Main collections:

- `content/notes/`
- `content/experiments/`
- `content/builds/`
- `content/journal/`
- `content/now/current.mdx`

2. Content Loading Layer

`lib/content.ts` is responsible for loading and preparing content.

It:

- Reads local `.mdx` files from disk
- Parses frontmatter using `gray-matter`
- Compiles MDX content
- Returns structured metadata and content objects
- Sorts entries by date in descending order

The main collections currently defined in `lib/content.ts` are:

- `experiments`
- `builds`
- `notes`

3. Routing Layer

The App Router renders both collection pages and individual entry pages.

Collection pages:

- `app/notes/page.tsx`
- `app/experiments/page.tsx`
- `app/builds/page.tsx`

Dynamic entry pages:

- `app/notes/[slug]/page.tsx`
- `app/experiments/[slug]/page.tsx`
- `app/builds/[slug]/page.tsx`

The Now page:

- `app/now/page.tsx`
- Renders `content/now/current.mdx`

4. Rendering Layer

MDX is compiled server-side and rendered inside App Router pages.

Entry metadata is generated from frontmatter.

The home page loads the latest experiment, latest build, and latest note, then displays them as cards using the shared `ContentCard` component.

5. Site Shell

The global site shell is handled through:

- `app/layout.tsx`
 - `components/Header.tsx`
 - `components/Footer.tsx`
-

Content Collections

Notes

Location:

`content/notes/`

Purpose:

Evergreen notes, ideas, concepts, technical explanations, and reflections that remain useful over time.

Examples:

- `learning-by-building.mdx`
- `curio-note-001-building-my-first-local-ai-lab.mdx`

Use notes for:

- Concepts
 - Technical explanations
 - Lessons learned
 - Mental models
 - Reusable references
-

Experiments

Location:

`content/experiments/`

Purpose:

Structured experiment logs that capture what was tested, why it was tested, what happened, and what was learned.

Examples:

- `local-ai-lab-001-first-n8n-ollama-test.mdx`
- `local-ai-lab-002-first-n8n-ollama-workflow.mdx`

Use experiments for:

- AI workflow tests
- Automation tests
- Tool comparisons
- Technical troubleshooting
- Small build attempts
- Hypothesis-driven learning

Recommended experiment structure:

```
---
title:
description:
date:
status:
tags:
---

## Question

## Hypothesis

## Setup

## Steps

## Result

## What Worked

## What Failed

## Lessons Learned

## Next Experiment
```

Builds

Location:

```
content/builds/
```

Purpose:

Project and build logs for larger efforts that may span multiple sessions.

Use builds for:

- Qurio site development
- Home lab setup
- Local AI lab
- Automation systems
- Learning platform prototypes
- Tools and products

Recommended build structure:

```
---
title:
description:
date:
status:
tags:
---

## Purpose

## Problem

## Approach

## Architecture

## Decisions

## Progress

## Lessons Learned

## Next Steps
```

Journal

Location:

content/journal/

Purpose:

Monthly or periodic reflection entries.

Current example:

- 2026-06.mdx

Use journal entries for:

- Progress reviews
- Personal reflections
- Weekly/monthly learning summaries
- Direction changes
- What is working
- What needs improvement

Recommended journal structure:

```
---
title:
description:
date:
status:
tags:
---

## What I Worked On

## What I Learned

## What Changed

## Wins

## Challenges

## Next Focus
```


Now Page

Location:

```
content/now/current.mdx
```

Purpose:

A snapshot of current focus, priorities, and active explorations.

The Now page should answer:

- What am I focused on now?
 - What am I learning?
 - What am I building?
 - What is paused?
 - What is next?
-

Frontmatter Standard

The app expects each content entry to use the following frontmatter fields:

```
---
title:
description:
date:
status:
tags:
---
```

Recommended conventions:

```
---
title: "Building My First Local AI Lab"
description: "Notes from setting up Ollama, n8n, and Docker for a local AI workflow."
date: "2026-06-10"
status: "active"
tags:
  - local-ai
  - docker
  - ollama
```

```
- n8n
---
```

Recommended status values:

- idea
- active
- complete
- paused
- archived

Current Content Inventory

Experiments

Current examples:

- local-ai-lab-001-first-n8n-ollama-test.mdx
- local-ai-lab-002-first-n8n-ollama-workflow.mdx

Notes

Current examples:

- learning-by-building.mdx
- curio-note-001-building-my-first-local-ai-lab.mdx

Journal

Current example:

- 2026-06.mdx

Now Page

Current file:

- current.mdx

Home Page Behavior

The home page should act as a simple entry point into the latest work.

It currently:

- Loads the latest experiment
- Loads the latest build
- Loads the latest note
- Displays them as cards using `ContentCard`

The home page should communicate:

- What Qurio is
- What is currently being explored
- What was recently learned
- Where a reader can start

Design Intent

Qurio should prioritize reading and clarity.

Design principles:

- Content first
- Simple navigation
- Calm layout
- Strong typography
- Low visual noise
- Clear metadata
- Easy scanning
- No hype-heavy language
- No fake authority
- No over-polished startup feel

The site should feel like a thoughtful builder's notebook, not a marketing funnel.

Development Commands

```
npm run dev
```

Runs the local development server.

```
npm run build
```

Creates the production build.

```
npm run typecheck
```

Runs TypeScript checks.

Current Project Priorities

1. Clean Content Structure

Make sure all content files follow the same frontmatter standard.

2. Improve Notes

Turn scattered learning into reusable notes.

Immediate notes to clean up:

- Learning by Building
- Building My First Local AI Lab
- Docker Basics
- Ollama Basics
- n8n Workflow Basics

3. Improve Experiments

Make each experiment follow a consistent structure:

- Question
- Hypothesis
- Setup
- Steps
- Result
- Lessons
- Next Experiment

4. Add Build Logs

Create build entries for larger efforts, starting with:

- Qurio Site Build
- Local AI Lab Build
- Home Server Learning Lab

5. Add Journal Rhythm

Use monthly journal entries to reflect on:

- What was built
 - What was learned
 - What improved
 - What is next
-

Near-Term Roadmap

Phase 1: Foundation

- Clean repo structure
- Standardize frontmatter
- Standardize note, experiment, build, and journal formats
- Improve README
- Improve docs

Phase 2: Content Quality

- Add better summaries
- Add tags consistently
- Add clearer statuses
- Add learning-focused conclusions
- Add cross-links between related notes and experiments

Phase 3: Site Experience

- Improve collection pages
- Add tag pages
- Add status filtering
- Add better MDX styling
- Add reading time
- Add previous/next navigation

Phase 4: Publishing System

- Create reusable content templates
 - Add content validation
 - Add a content checklist
 - Add GitHub issue templates for new notes and experiments
-

Recommended Content Rules

Every meaningful Qurio entry should answer:

1. What was I trying to learn?
2. What did I try?
3. What happened?
4. What did I learn?
5. What would I do next?

Every experiment should leave behind an artifact.

Every artifact should help the future version of me or another curious builder understand the process.

Working Philosophy

Qurio should operate as a practical learning system.

The preferred loop is:

Question → Experiment → Build → Document → Reflect → Improve

This keeps the project grounded in action instead of passive learning.

The goal is not to know everything before starting.

The goal is to start, learn from the attempt, and improve through repetition.

AI Assistant Guidance for Qurio

When helping with Qurio, the assistant should:

- Explain the reasoning behind recommendations
- Teach while solving
- Prefer practical implementation over abstract theory
- Follow industry best practices
- Reference official documentation when relevant
- Encourage documentation after meaningful learning
- Help turn chats into durable notes, experiments, and build logs
- Keep language honest, calm, and practical
- Avoid hype-heavy, guru-style, or get-rich-quick framing

For technical tasks, use this structure when helpful:

1. Objective
2. Why it matters
3. Concepts
4. Implementation
5. Best practices
6. Documentation references
7. Lessons learned
8. Next experiment

Current Best Next Step

Clean up the content system before adding more features.

Recommended next task:

Standardize the existing `.mdx` files so that notes, experiments, builds, journal entries, and the Now page all follow consistent formats.

Start with:

1. `content/experiments/local-ai-lab-001-first-n8n-ollama-test.mdx`
2. `content/experiments/local-ai-lab-002-first-n8n-ollama-workflow.mdx`
3. `content/notes/learning-by-building.mdx`
4. `content/notes/curio-note-001-building-my-first-local-ai-lab.mdx`
5. `content/journal/2026-06.mdx`
6. `content/now/current.mdx`

This will make Curio easier to maintain, easier to publish, and easier to expand.